Chapter 8 CRT

1. Explain the difference between a has-a and is-a relationship among classes.
An is-a relationship occurs when one class inherits from another class. This means the derived class is a more specific version of the base class. For example, a Dog is an Animal.
A has-a relationship occurs when one class contains an object of another class as a component. For example, a Car has an Engine. The main difference is that is-a represents inheritance, while has-a represents composition.

2. If a base class has a public method go() and a derived class has a public method stop(), which methods will be available to an object of the derived class?
Both go() and stop() will be available to an object of the derived class. This is because the derived class inherits accessible public methods from its base class while also having access to its own methods. Therefore, the object can use both methods.

3. Compare and contrast implementing an abstract method to overriding a method.
Implementing an abstract method means providing the actual code for a method that was declared without a body in an abstract class or interface. Overriding a method means replacing an inherited method's implementation with a new version in a derived class. Both involve defining how a method behaves in a subclass. However, implementing an abstract method is required for a concrete subclass to be complete, while overriding an existing method is optional unless the subclass needs different behaviour.

4. Compare and contrast an abstract class to an interface.
An abstract class is a class that cannot be instantiated directly and can contain both completed methods and abstract methods. It can also contain instance variables and constructors. A class uses extends to inherit from an abstract class. An interface defines a set of methods that a class agrees to implement. It is used to establish common behaviour between potentially unrelated classes. A class uses implements to follow an interface. Both support abstraction and help organize code. However, a Java class can only extend one class, but it can implement multiple interfaces.

6. Use the following classes to answer the questions below.
a) What type of method is doThat() in Wo?
doThat() is an abstract method because it is declared in an interface without a method body. It specifies what a class must implement without providing the actual instructions for how the method works.
b) What is Wo?
Wo is an interface. It acts as a blueprint that requires any non-abstract class implementing it to provide an implementation of its declared abstract methods, such as doThat().
c) Why is doThat() implemented in Roo?
Roo implements the Wo interface, meaning it is required to provide an implementation of the doThat() method. This allows Roo to satisfy the requirements of the interface.

d) List the methods available to a Roo object.

A Roo object has access to the following public methods:

1. doThis() – returns 10.
2. doNow() – returns 15.
3. doThat() – returns 20 (once its access modifier is corrected to public).

The doThis() method inherited from Bo is overridden by Roo, so the object uses the version defined in Roo.

e) How does the implementation of doThis() in Roo affect the implementation of doThis() in Bo?

The doThis() method in Roo overrides the method in Bo. This means that when a doThis() call is made using a Roo object, Java executes the version in Roo, which returns 10 instead of the original value of 2 from Bo. The original method in Bo still exists, but it is replaced by the overridden version when called normally through a Roo object.

f) What action does the statement super(1) in Roo perform?

super(1) calls the constructor of the parent class, Bo, and passes it the integer value 1. This initializes the inherited instance variable x to 1 through the Bo constructor.

g) Can the doThis() method in Bo be called from a Roo object? If so, how?

Yes. The doThis() method from Bo can still be accessed by a Roo object using the super keyword from within the Roo class. However, calling super.doThis() directly from outside the class is not possible.

h) Can a method in Roo call the doThis() method in Bo? If so, how?

Yes. A method inside Roo can call the original doThis() method from Bo by using the super keyword.

For example:

```
public int callParentMethod() {
   return super.doThis();
}
```

This calls the original method from Bo, which returns 2, rather than the overridden method in Roo, which returns 10.